

 CLSweet / eCFR-Analyzer-Case-Study Public[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#)**eCFR-Analyzer-Case-Study / app.py**

...

 Cannot retrieve latest commit at this time.

1378 lines (1123 loc) · 62.3 KB

Code BlameRaw     

```
1  import streamlit as st
2  import requests
3  import pandas as pd
4  import numpy as np
5  import re
6  import matplotlib.pyplot as plt
7  import time
8  from datetime import datetime, timedelta
9  from collections import defaultdict
10 from bs4 import BeautifulSoup
11 import plotly.graph_objects as go
12 import plotly.express as px
13 import networkx as nx
14
15 # Define global variables with default values
16 BASE_URL = "https://www.ecfr.gov"
17 request_timeout = 240 # Default value
18 skip_problematic_titles = True # Default value
19 throttle_delay = 0.2 # Default value
20 cache_results=True
21
22 # Set page config
23 st.set_page_config(
24     page_title="Federal Regulations Word Count Analysis Tool",
25     page_icon="🇺🇸",
26     layout="wide",
27     initial_sidebar_state="expanded"
28 )
29
30 # Functions to interact with the eCFR API
31 def get_agencies(force_refresh=False):
32     """Fetch all agencies from the API with caching support"""
```

```
33     cache_key = "agencies_cache"
34
35     # Check if we have cached data and caching is enabled
36     if not force_refresh and cache_results and cache_key in st.session_state:
37         return st.session_state[cache_key]
38
39     # If no cache or forced refresh, fetch from API
40     url = f"{BASE_URL}/api/admin/v1/agencies.json"
41     try:
42         response = requests.get(url, timeout=request_timeout)
43         if response.status_code == 200:
44             agencies = response.json().get("agencies", [])
45             # Cache the result if caching is enabled
46             if cache_results:
47                 st.session_state[cache_key] = agencies
48             return agencies
49         else:
50             st.error(f"Error fetching agencies: {response.status_code}")
51             return []
52     except requests.exceptions.Timeout:
53         st.error("Request timed out when fetching agencies. Try increasing the timeout.")
54         return []
55     except Exception as e:
56         st.error(f"Error fetching agencies: {str(e)}")
57         return []
58
59     def get_titles(force_refresh=False):
60         """Fetch summary information about all titles with caching support"""
61         cache_key = "titles_cache"
62
63         # Check if we have cached data and caching is enabled
64         if not force_refresh and cache_results and cache_key in st.session_state:
65             return st.session_state[cache_key]
66
67         # If no cache or forced refresh, fetch from API
68         url = f"{BASE_URL}/api/versioner/v1/titles.json"
69         try:
70             response = requests.get(url, timeout=request_timeout)
71             if response.status_code == 200:
72                 titles = response.json().get("titles", [])
73                 # Cache the result if caching is enabled
74                 if cache_results:
75                     st.session_state[cache_key] = titles
76                 return titles
77             else:
78                 st.error(f"Error fetching titles: {response.status_code}")
79                 return []
80         except requests.exceptions.Timeout:
81             st.error("Request timed out when fetching titles. Try increasing the timeout.")
```

```
82         return []
83     except Exception as e:
84         st.error(f"Error fetching titles: {str(e)}")
85         return []
86
87     def get_title_content(title_number, date, max_retries=1, force_refresh=False):
88         """Fetch XML content for a title on a specific date with retry capability and caching"""
89         cache_key = f"title_content_{title_number}_{date}"
90
91         # Check if we have cached data and caching is enabled
92         if not force_refresh and cache_results and cache_key in st.session_state:
93             return st.session_state[cache_key]
94
95         # Skip known problematic titles if option is enabled
96         if skip_problematic_titles and title_number in [7, 10, 40, 42, 45]:
97             st.warning(f"Skipping Title {title_number} as it's known to cause timeouts due to its")
98             return None
99
100        url = f"{BASE_URL}/api/versioner/v1/full/{date}/title-{title_number}.xml"
101
102        retries = 0
103        while retries <= max_retries:
104            try:
105                response = requests.get(url, timeout=request_timeout)
106                if response.status_code == 200:
107                    content = response.text
108                    # Cache the result if caching is enabled
109                    if cache_results:
110                        st.session_state[cache_key] = content
111                    return content
112                elif response.status_code == 504:
113                    retries += 1
114                    if retries <= max_retries:
115                        st.warning(f"Gateway timeout (504) for Title {title_number}. Retrying ({retries})")
116                        time.sleep(throttle_delay * 2) # Wait longer between retries
117                    else:
118                        st.warning(f"Gateway timeout (504) for Title {title_number} after {max_retries} retries")
119                        return None
120                elif response.status_code == 404:
121                    st.warning(f"Title {title_number} not available for date {date}")
122                    return None
123                else:
124                    st.error(f"Error fetching title {title_number} content: {response.status_code}")
125                    return None
126            except requests.exceptions.Timeout:
127                retries += 1
128                if retries <= max_retries:
129                    st.warning(f"Request timed out for Title {title_number}. Retrying ({retries})")
130                    time.sleep(throttle_delay * 2)
```

```
131         else:
132             st.warning(f"Request timed out for Title {title_number} after {max_retries} re
133             return None
134     except Exception as e:
135         st.error(f"Error fetching title {title_number} content: {str(e)}")
136         return None
137
138     ✓ def count_words_in_xml(xml_content):
139         """Count words in XML content using BeautifulSoup"""
140         if not xml_content:
141             return 0
142
143         try:
144             # Parse XML with BeautifulSoup
145             soup = BeautifulSoup(xml_content, 'lxml-xml')
146
147             # Extract all text
148             text = soup.get_text()
149
150             # Count words
151             words = re.findall(r'\b\w+\b', text)
152             return len(words)
153         except Exception as e:
154             st.warning(f"Error parsing XML: {str(e)}")
155             # Fall back to simple regex approach
156             text_without_tags = re.sub(r'<[^>]+>', ' ', xml_content)
157             words = re.findall(r'\b\w+\b', text_without_tags)
158             return len(words)
159
160     ✓ def create_agency_title_mapping(agency):
161         """Create a mapping of agencies to the titles they regulate"""
162         agency_to_titles = defaultdict(set)
163
164         for agency in agencies:
165             agency_name = agency["name"]
166
167             # Add titles directly regulated by this agency
168             if "cfr_references" in agency:
169                 for ref in agency["cfr_references"]:
170                     if "title" in ref:
171                         agency_to_titles[agency_name].add(ref["title"])
172
173             # Process child agencies
174             if "children" in agency and agency["children"]:
175                 for child in agency["children"]:
176                     child_name = child["name"]
177                     if "cfr_references" in child:
178                         for ref in child["cfr_references"]:
179                             if "title" in ref:
```

```
180         agency_to_titles[child_name].add(ref["title"])
181
182     return agency_to_titles
183
184     def extract_agency_hierarchy(agency):
185         """Extract the hierarchy of agencies for visualization"""
186         nodes = []
187         edges = []
188
189         # Function to process an agency and its children
190     def process_agency(agency, parent=None):
191         agency_id = agency["slug"]
192         agency_name = agency["name"]
193
194         # Add node
195         nodes.append({
196             "id": agency_id,
197             "name": agency_name,
198             "short_name": agency.get("short_name", ""),
199             "is_parent": len(agency.get("children", [])) > 0
200         })
201
202         # Add edge if there is a parent
203         if parent:
204             edges.append((parent, agency_id))
205
206         # Process children
207         for child in agency.get("children", []):
208             process_agency(child, agency_id)
209
210         # Process all top-level agencies
211         for agency in agencies:
212             process_agency(agency)
213
214         return nodes, edges
215
216     def create_agency_hierarchy_graph(agencies):
217         """Create a NetworkX graph for agency hierarchy"""
218         nodes, edges = extract_agency_hierarchy(agencies)
219
220         G = nx.DiGraph()
221
222         # Add nodes with attributes
223         for node in nodes:
224             G.add_node(node["id"],
225                         name=node["name"],
226                         short_name=node["short_name"],
227                         is_parent=node["is_parent"])
228
```

```
229     # Add edges
230     G.add_edges_from(edges)
231
232     return G, nodes
233
234 ✓ def calculate_word_counts_over_time(agencies, titles_info, years, max_titles, throttle_delay)
235     """Calculate word counts for each agency over multiple years"""
236     # Get agency to title mapping
237     agency_to_titles = create_agency_title_mapping(agencies)
238
239     # Create reverse mapping (title -> agencies)
240     title_to_agencies = defaultdict(list)
241     for agency, titles in agency_to_titles.items():
242         for title in titles:
243             title_to_agencies[title].append(agency)
244
245     # Dictionary to store word counts by year and agency
246     word_counts_by_year = {}
247
248     # Process titles for each year
249     for year in years:
250         target_date = f"{year}-01-01"
251         st.write(f"Processing year: {year}")
252
253         progress_bar = st.progress(0)
254
255         # Dictionary to store word counts by title for this year
256         title_word_counts = {}
257
258         # Process titles (limited to max_titles)
259         process_size = min(max_titles, len(titles_info))
260
261         for i, title_info in enumerate(titles_info[:process_size]):
262             title_number = title_info["number"]
263             title_name = title_info.get("name", "Unknown")
264             latest_date = title_info["latest_amended_on"]
265
266             # Update progress
267             progress = (i + 1) / process_size
268             progress_bar.progress(progress)
269
270             # Skip reserved titles
271             if title_info.get("reserved", False):
272                 continue
273
274             # Determine date to use
275             use_date = target_date
276             if datetime.strptime(latest_date, "%Y-%m-%d") < datetime.strptime(target_date, "%Y-%m-%d"):
277                 use_date = latest_date
```

```
278
279     # Fetch and process content
280     title_content = get_title_content(title_number, use_date)
281
282     if title_content:
283         word_count = count_words_in_xml(title_content)
284         title_word_counts[title_number] = word_count
285
286     # Add delay to avoid rate limiting
287     time.sleep(throttle_delay)
288
289     # Calculate word counts per agency for this year
290     agency_word_counts = defaultdict(int)
291     for title, word_count in title_word_counts.items():
292         agencies_for_title = title_to_agencies.get(title, [])
293         if agencies_for_title:
294             # If a title is regulated by multiple agencies, distribute the word count
295             count_per_agency = word_count / len(agencies_for_title)
296             for agency in agencies_for_title:
297                 agency_word_counts[agency] += count_per_agency
298
299     # Store results for this year
300     word_counts_by_year[year] = dict(agency_word_counts)
301
302     progress_bar.empty()
303
304     return word_counts_by_year
305
306 ✓ def main():
307     global request_timeout, skip_problematic_titles, throttle_delay, cache_results
308
309     st.title("Code of Federal Regulations Analyzer")
310     st.write("""
311     This app analyzes the word count of federal regulations per agency using the Electronic Code of Federal Regulations (eCFR) API.
312     """)
313
314     # Sidebar configuration
315     st.sidebar.header("Analysis Configuration")
316
317     # Date input with default of 2025-01-01
318     default_date = datetime(2025, 1, 1)
319     target_date = st.sidebar.date_input("Select a Date Cutoff", value=default_date)
320     target_date_str = target_date.strftime("%Y-%m-%d")
321
322     # Sample size selector
323     max_titles = st.sidebar.slider("Number of titles to process", 1, 50, 5,
324                                     help="Higher values provide more complete results but take longer to process.")
325
326     # Advanced options
```

```

327 with st.sidebar.expander("Advanced Options"):
328     cache_results = st.checkbox("Cache results", value=True,
329                                 help="Store results to avoid reprocessing if parameters don't change")
330     throttle_delay = st.slider("API request delay (seconds)", 0.1, 2.0, 0.2, 0.1,
331                                help="Increase to avoid API rate limiting")
332     request_timeout = st.slider("Request timeout (seconds)", 30, 600, 240, 30,
333                                 help="Maximum time to wait for API response before timing out")
334     skip_problematic_titles = st.checkbox("Skip known large titles", value=True,
335                                           help="Skip titles known to cause timeouts (7, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375)
337 # Analytics tabs
338 tab1, tab2, tab3, tab4, tab5, tab6 = st.tabs(["Agency Analysis", "Title Analysis", "Agency Word Counts", "Title Word Counts", "Agency Word Counts by Agency", "Title Word Counts by Agency"])
339
340 with tab1:
341     st.header("Regulation Word Count by Agency")
342
343     if st.button("Calculate Word Counts", type="primary"):
344         # Create a cache key based on the parameters
345         cache_key = f"agency_analysis_{max_titles}_{target_date_str}_{skip_problematic_titles}"
346
347         # Check if we have cached results and caching is enabled
348         if cache_results and cache_key in st.session_state:
349             st.success("Using cached results. Toggle 'Cache results' off to force recalculation.")
350
351             # Retrieve cached data
352             agencies = st.session_state[f"{cache_key}_agencies"]
353             titles_info = st.session_state[f"{cache_key}_titles_info"]
354             title_word_counts = st.session_state[f"{cache_key}_title_word_counts"]
355             df = st.session_state[f"{cache_key}_all_agencies_df"]
356             senior_df = st.session_state[f"{cache_key}_senior_agencies_df"]
357             title_df = st.session_state[f"{cache_key}_title_df"]
358             agency_word_counts = st.session_state[f"{cache_key}_agency_word_counts"]
359
360             # Store in session state for other tabs
361             st.session_state.agencies = agencies
362             st.session_state.titles_info = titles_info
363             st.session_state.title_df = title_df
364             st.session_state.title_word_counts = title_word_counts
365             st.session_state.agency_word_counts = agency_word_counts
366             st.session_state.all_agencies_df = df
367             st.session_state.senior_agencies_df = senior_df
368         else:
369             # Initialize progress tracking
370             progress_bar = st.progress(0)
371             status_text = st.empty()
372             results_placeholder = st.empty()
373
374             try:
375                 status_text.text("Fetching agencies and titles...")

```



```

376 # Force refresh if cache disabled
377 agencies = get_agencies(force_refresh=not cache_results)
378 titles_info = get_titles(force_refresh=not cache_results)
379
380 # Store in session state for other tabs
381 st.session_state.agencies = agencies
382 st.session_state.titles_info = titles_info
383
384 # Create agency-title mappings
385 agency_to_titles = create_agency_title_mapping(agencies)
386 title_to_agencies = defaultdict(list)
387 for agency, titles in agency_to_titles.items():
388     for title in titles:
389         title_to_agencies[title].append(agency)
390
391 # Identify most senior agencies (those with no parent)
392 all_child_names = set()
393 for agency in agencies:
394     for child in agency.get("children", []):
395         all_child_names.add(child["name"])
396
397 # Process titles
398 title_word_counts = {}
399 process_size = min(max_titles, len(titles_info))
400
401 col1, col2 = st.columns(2)
402 with col1:
403     st.subheader("Processing Status")
404     processing_status = st.empty()
405
406 with col2:
407     st.subheader("Current Statistics")
408     stats_display = st.empty()
409
410 for i, title_info in enumerate(titles_info[:process_size]):
411     title_number = title_info["number"]
412     title_name = title_info.get("name", "Unknown")
413     latest_date = title_info["latest_amended_on"]
414
415     # Update progress
416     progress = (i + 1) / process_size
417     progress_bar.progress(progress)
418     processing_status.write(f"Processing title {i+1} of {process_size}: T
419
420     # Skip reserved titles
421     if title_info.get("reserved", False):
422         processing_status.write(f"Skipping reserved title {title_number}")
423         continue
424
425     # Determine date to use

```

```

425         # Determine date to use
426         use_date = target_date_str
427         if datetime.strptime(latest_date, "%Y-%m-%d") < datetime.strptime(target_date, "%Y-%m-%d"):
428             use_date = latest_date
429             processing_status.write(f"Using latest available date ({latest_date})")
430
431         # Fetch and process content with cache control
432         title_content = get_title_content(title_number, use_date, force_refresh=True)
433
434         if title_content:
435             word_count = count_words_in_xml(title_content)
436             title_word_counts[title_number] = word_count
437
438         # Update stats
439         stats_display.write(f"""
440         **Title {title_number}**
441         - Name: {title_name}
442         - Word count: {word_count:,}
443         - Date analyzed: {use_date}
444         """)
445
446         # Add delay to avoid rate limiting
447         time.sleep(throttle_delay)
448
449         # Calculate word counts per agency
450         agency_word_counts = defaultdict(int)
451         for title, word_count in title_word_counts.items():
452             agencies_for_title = title_to_agencies.get(title, [])
453             if agencies_for_title:
454                 # If a title is regulated by multiple agencies, distribute the word count
455                 count_per_agency = word_count / len(agencies_for_title)
456                 for agency in agencies_for_title:
457                     agency_word_counts[agency] += count_per_agency
458
459         # Create DataFrame with all agencies
460         df = pd.DataFrame(list(agency_word_counts.items()), columns=["Agency", "Word Count"])
461         df["Word Count"] = df["Word Count"].astype(int)
462         df = df.sort_values(by="Word Count", ascending=False).reset_index(drop=True)
463
464         # Create a DataFrame with only top-level (senior) agencies
465         senior_agencies = [agency["name"] for agency in agencies]
466         senior_df = df[df["Agency"].isin(senior_agencies)]
467         senior_df = senior_df.sort_values(by="Word Count", ascending=False).reset_index(drop=True)
468
469         # Create title word count DataFrame
470         title_df = pd.DataFrame({
471             "Title": [f"Title {title}" for title in title_word_counts.keys()],
472             "Word Count": list(title_word_counts.values())
473         })
474         title_df = title_df.sort_values(by="Word Count", ascending=False).reset_index(drop=True)

```

```

474         title_df = title_df.sort_values(by= word_count , ascending=False).reset_index()
475
476     # Store data in session state
477     st.session_state.agency_word_counts = agency_word_counts
478     st.session_state.all_agencies_df = df
479     st.session_state.senior_agencies_df = senior_df
480     st.session_state.title_df = title_df
481     st.session_state.title_word_counts = title_word_counts
482
483     # Cache results if enabled
484     if cache_results:
485         st.session_state[f"{cache_key}_agencies"] = agencies
486         st.session_state[f"{cache_key}_titles_info"] = titles_info
487         st.session_state[f"{cache_key}_title_word_counts"] = title_word_counts
488         st.session_state[f"{cache_key}_all_agencies_df"] = df
489         st.session_state[f"{cache_key}_senior_agencies_df"] = senior_df
490         st.session_state[f"{cache_key}_title_df"] = title_df
491         st.session_state[f"{cache_key}_agency_word_counts"] = agency_word_counts
492
493     # Clear placeholders
494     progress_bar.empty()
495     status_text.empty()
496     processing_status.empty()
497     stats_display.empty()
498
499     # Display results
500     results_placeholder.write("### Results")
501
502     # Add tabs for different views
503     results_tab1, results_tab2 = st.tabs(["Senior Agencies", "All Agencies"])
504
505     with results_tab1:
506         st.write(f"Showing {len(senior_df)} Agencies")
507         st.dataframe(senior_df, use_container_width=True)
508
509         # Visualization for senior agencies only
510         st.subheader("Top 10 Agencies by Word Count")
511         fig, ax = plt.subplots(figsize=(12, 6))
512         top_df = senior_df.head(10) if len(senior_df) > 10 else senior_df
513         bars = ax.bar(top_df["Agency"], top_df["Word Count"])
514
515         # Calculate the maximum bar height for y-axis adjustment
516         max_height = top_df["Word Count"].max()
517
518         # Add value labels on top of each bar
519         for bar in bars:
520             height = bar.get_height()
521             ax.text(bar.get_x() + bar.get_width()/2., height + max_height*0.05,
522                    f'{height:,}', ha='center', va='bottom', rotation=0)
523
524

```

```

524         # Set y-axis limit with 15% padding to avoid clipping
525         ax.set_ylim(0, max_height * 1.15)
526
527         plt.xticks(rotation=45, ha="right")
528         plt.title(f"Top Agencies by Regulation Word Count (as of {target_date_
529         plt.ylabel("Word Count")
530         plt.tight_layout()
531         st.pyplot(fig)
532
533         with results_tab2:
534             st.write(f"Showing all {len(df)} agencies (including child agencies)"
535             st.dataframe(df, use_container_width=True)
536
537         # Download button for results
538         csv = df.to_csv(index=False)
539         st.download_button(
540             label="Download complete agency data as CSV",
541             data=csv,
542             file_name=f"regulation_wordcount_by_agency_{target_date_str}.csv",
543             mime="text/csv"
544         )
545
546         # Store title data for the second tab
547         st.session_state.title_df = title_df
548         st.session_state.title_word_counts = title_word_counts
549
550         except Exception as e:
551             st.error(f"An error occurred: {str(e)}")
552             st.write("This could be due to API rate limits or unavailability. Please
553
554     with tab2:
555         st.header("Title-Level Analysis")
556
557         if 'title_df' in st.session_state:
558             st.subheader("Word Count by Title")
559             st.dataframe(st.session_state.title_df, use_container_width=True)
560
561             # Visualization for titles
562             st.subheader("Top 10 Titles by Word Count")
563             fig, ax = plt.subplots(figsize=(12, 6))
564             top_title_df = st.session_state.title_df.head(10)
565             bars = ax.bar(top_title_df["Title"], top_title_df["Word Count"])
566
567             # Add value labels
568             for bar in bars:
569                 height = bar.get_height()
570                 ax.text(bar.get_x() + bar.get_width()/2., height + 5,
571                     f'{height:,}', ha='center', va='bottom', rotation=0)
572

```

```
573         plt.xticks(rotation=45, ha="right")
574         plt.title(f"Top Titles by Word Count (as of {target_date_str})")
575         plt.ylabel("Word Count")
576         plt.tight_layout()
577         st.pyplot(fig)
578
579         # Download button for title data
580         csv = st.session_state.title_df.to_csv(index=False)
581         st.download_button(
582             label="Download title data as CSV",
583             data=csv,
584             file_name=f"regulation_wordcount_by_title_{target_date_str}.csv",
585             mime="text/csv"
586         )
587     else:
588         st.info("Please run the analysis in the 'Agency Analysis' tab first to view title
589
590 with tab3:
591     st.header("Agency Hierarchy Visualization")
592
593     # Check if agencies data is available or fetch it
594     if 'agencies' not in st.session_state:
595         with st.spinner("Fetching agency data..."):
596             st.session_state.agencies = get_agencies()
597
598     if st.session_state.agencies:
599         # Get top-level agencies (most senior parents)
600         top_level_agencies = [a for a in st.session_state.agencies]
601
602         # Find agencies with children (parents)
603         parent_agencies = []
604
605         # Function to find parent agencies recursively
606     def find_parent_agencies(agency_list):
607         for agency in agency_list:
608             # Check if this agency has children
609             if agency.get("children", []) and len(agency["children"]) > 0:
610                 parent_agencies.append(agency)
611             # Also check children recursively for any that might be parents
612             find_parent_agencies(agency["children"])
613
614         # Process all agencies to find parent ones
615         find_parent_agencies(top_level_agencies)
616
617         # Find independent agencies (those not listed as children of any other agency)
618         all_child_slugs = set()
619         for agency in st.session_state.agencies:
620             for child in agency.get("children", []):
621                 all_child_slugs.add(child["slug"])
```

```
622
623     # Collect independent agencies that are not top-level
624     independent_agencies = []
625
626     # Function to find independent agencies recursively
627     def find_independent_agencies(agency_list):
628         for agency in agency_list:
629             # Check if this agency has children
630             if agency.get("children", []):
631                 # Check children recursively
632                 find_independent_agencies(agency["children"])
633             # If this agency appears in top level but isn't a child of anyone, it's independent
634             elif agency["slug"] not in all_child_slugs:
635                 independent_agencies.append(agency)
636
637     # Process all agencies to find independent ones
638     find_independent_agencies(top_level_agencies)
639
640     # Display agency counts
641     total_agencies = sum(1 for a in top_level_agencies) + sum(len(a.get("children", [])) for a in top_level_agencies)
642     st.write(f"Total Agencies: {total_agencies}")
643     st.write(f"Parent Agencies: {len(parent_agencies)}")
644     st.write(f"Independent Agencies (No Parent): {len(independent_agencies)}")
645
646     # Create dropdown options - only parent agencies (those with children) and an option for all agencies
647     dropdown_options = ["All Agencies", "Agencies Without Parent"] + [a["name"] for a in parent_agencies]
648
649     selected_option = st.selectbox(
650         "Filter by agency category:",
651         dropdown_options
652     )
653
654     # Create a function to build a hierarchical sunburst chart
655     def build_sunburst_data(agencies, independent_agencies, parent_agencies, selected_option):
656         data = []
657
658         # Add root node
659         data.append({
660             "id": "root",
661             "parent": "",
662             "name": "Federal Agencies",
663             "value": 1
664         })
665
666         # Process each agency
667         def process_agency(agency, parent="root", include=True):
668             if not include:
669                 return
670             # Add agency node
```

```
671         agency_id = agency["slug"]
672         agency_name = agency["name"]
673
674         # Add this agency
675         data.append({
676             "id": agency_id,
677             "parent": parent,
678             "name": agency_name,
679             "value": 1
680         })
681
682         # Add children
683         for child in agency.get("children", []):
684             process_agency(child, agency_id, include)
685
686         # Handle different filter options
687         if selected_option == "All Agencies":
688             # Include all agencies
689             for agency in agencies:
690                 process_agency(agency)
691
692         elif selected_option == "Agencies Without Parent":
693             # Create a virtual parent for independent agencies
694             data.append({
695                 "id": "independent",
696                 "parent": "root",
697                 "name": "Agencies Without Parent",
698                 "value": 1
699             })
700
701             # Add all independent agencies under this virtual parent
702             for agency in independent_agencies:
703                 agency_id = agency["slug"]
704                 agency_name = agency["name"]
705
706                 data.append({
707                     "id": agency_id,
708                     "parent": "independent",
709                     "name": agency_name,
710                     "value": 1
711                 })
712
713         else:
714             # Find the selected parent agency
715             selected_agency = next((a for a in parent_agencies if a["name"] == selected_option), None)
716             if selected_agency:
717                 process_agency(selected_agency)
718
719
```

```
720         return pd.DataFrame(data)
721
722     # Build the data based on selection
723     sunburst_df = build_sunburst_data(top_level_agencies, independent_agencies, parent
724
725     # Create the sunburst chart
726     fig = px.sunburst(
727         sunburst_df,
728         ids='id',
729         parents='parent',
730         names='name',
731         title=f"Agency Hierarchy: {selected_option}",
732         height=700
733     )
734
735     fig.update_layout(margin=dict(t=30, l=0, r=0, b=0))
736
737     st.plotly_chart(fig, use_container_width=True)
738
739     # Also provide a table view
740     st.subheader("Agency Table")
741
742     # Create a more readable table
743     table_data = []
744
745     def process_agency_for_table(agency, level=0):
746         agency_name = agency["name"]
747         agency_id = agency["slug"]
748
749         # Add this agency
750         table_data.append({
751             "Agency": " " * level + agency_name,
752             "Level": level,
753             "ID": agency_id,
754             "Short Name": agency.get("short_name", ""),
755             "Parent": "Yes" if agency.get("children", []) else "No"
756         })
757
758         # Add children
759         for child in sorted(agency.get("children", []), key=lambda x: x["name"]):
760             process_agency_for_table(child, level + 1)
761
762     # Process agencies based on selection
763     if selected_option == "All Agencies":
764         # Include all top-level agencies
765         for agency in sorted(top_level_agencies, key=lambda x: x["name"]):
766             process_agency_for_table(agency)
767
768     elif selected_option == "Agencies Without Parent":
```



```

769         # Include only independent agencies
770         for agency in sorted(independent_agencies, key=lambda x: x["name"]):
771             process_agency_for_table(agency, 0) # Level 0 since they're all top-level
772
773     else:
774         # Find and include only the selected parent agency
775         selected_agency = next((a for a in parent_agencies if a["name"] == selected_option), None)
776         if selected_agency:
777             process_agency_for_table(selected_agency)
778
779     # Create the table DataFrame
780     table_df = pd.DataFrame(table_data)
781     if not table_df.empty:
782         st.dataframe(table_df[["Agency", "Short Name", "Parent"]], use_container_width=True)
783
784         # Show count of agencies in this view
785         st.write(f"Showing {len(table_df)} agencies in this view")
786
787         # Add download button for the current view
788         csv = table_df.to_csv(index=False)
789         safe_filename = selected_option.replace(" ", "_").replace("/", "_")
790         st.download_button(
791             label="Download agency list as CSV",
792             data=csv,
793             file_name=f"agencies_{safe_filename}.csv",
794             mime="text/csv"
795         )
796     else:
797         st.info("No agencies to display with the current filter.")
798
799     else:
800         st.warning("No agency data available. Please run the analysis in the 'Agency Analysis' tab.")
801
802     with tab4:
803         st.header("Word Count Over Time")
804
805         # Year range selection
806         st.subheader("Select Years to Analyze")
807
808         col1, col2 = st.columns(2)
809         with col1:
810             start_year = st.number_input("Start Year", min_value=2010, max_value=2025, value=2010)
811         with col2:
812             end_year = st.number_input("End Year", min_value=2010, max_value=2025, value=2024)
813
814         if end_year < start_year:
815             st.error("End year must be greater than or equal to start year.")
816         else:
817             years_to_analyze = list(range(start_year, end_year + 1))

```

```

818
819     # Check if agencies data is available or fetch it
820     if 'agencies' not in st.session_state:
821         with st.spinner("Fetching agency data..."):
822             st.session_state.agencies = get_agencies(force_refresh=not cache_results)
823
824     # Get all agencies
825     if st.session_state.agencies:
826         # Extract all agency names
827         all_agency_names = []
828
829     def collect_agency_names(agency_list):
830         for agency in agency_list:
831             all_agency_names.append(agency["name"])
832             if "children" in agency and agency["children"]:
833                 collect_agency_names(agency["children"])
834
835     collect_agency_names(st.session_state.agencies)
836     all_agency_names = sorted(all_agency_names)
837
838     # Agency selection
839     st.subheader("Select Agencies to Analyze")
840
841     # Option to select all
842     select_all = st.checkbox("Select All Agencies", value=False)
843
844     if select_all:
845         selected_agencies = all_agency_names
846         st.info(f"All {len(all_agency_names)} agencies selected")
847     else:
848         # Multi-select for agencies
849         selected_agencies = st.multiselect(
850             "Choose agencies to analyze:",
851             options=all_agency_names,
852             default=all_agency_names[:5] if len(all_agency_names) >= 5 else all_ag
853         )
854
855     if not selected_agencies:
856         st.warning("Please select at least one agency to analyze.")
857     else:
858         st.info(f"{len(selected_agencies)} agencies selected")
859
860     if selected_agencies:
861         if st.button("Calculate Word Counts Over Time", type="primary"):
862             # Create a cache key based on the parameters
863             # Sort and join selected_agencies for a consistent key regardless of s
864             agencies_key = "_".join(sorted([agency.replace(" ", "_")[:10] for agen
865             cache_key = f"wordcount_time_{start_year}_{end_year}_{max_titles}_{ta
866

```

```
867         # Check if we have cached results and caching is enabled
868         if cache_results and cache_key in st.session_state:
869             st.success("Using cached results. Toggle 'Cache results' off to fo
870
871         # Retrieve cached data
872         word_counts_by_year = st.session_state[f"{cache_key}_word_counts_b
873         time_df = st.session_state[f"{cache_key}_time_df"]
874     else:
875         # Check if titles data is available or fetch it
876         if 'titles_info' not in st.session_state:
877             with st.spinner("Fetching titles data..."):
878                 st.session_state.titles_info = get_titles(force_refresh=ne
879
880         # Calculate word counts for each year
881         with st.spinner(f"Calculating word counts for years {start_year}--
882             word_counts_by_year = calculate_word_counts_over_time(
883                 st.session_state.agencies,
884                 st.session_state.titles_info,
885                 years_to_analyze,
886                 max_titles,
887                 throttle_delay
888             )
889
890         # Create DataFrame for plotting
891         plot_data = []
892
893         for year, agency_counts in word_counts_by_year.items():
894             for agency, count in agency_counts.items():
895                 if agency in selected_agencies:
896                     plot_data.append({
897                         "Year": year,
898                         "Agency": agency,
899                         "Word Count": int(count)
900                     })
901
902         time_df = pd.DataFrame(plot_data)
903
904         # Cache results if enabled
905         if cache_results:
906             st.session_state[f"{cache_key}_word_counts_by_year"] = word_co
907             st.session_state[f"{cache_key}_time_df"] = time_df
908             # Store the cache key itself for reference
909             st.session_state[cache_key] = True
910
911         # Store in session state for potential use in other tabs
912         st.session_state.word_counts_by_year = word_counts_by_year
913         st.session_state.time_df = time_df
914
915         if time_df.empty:
```

```
916         st.warning("No data available for the selected agencies and years")
917     else:
918         # Create the interactive line chart for all selected agencies
919         st.subheader(f"Word Count Trends for {len(selected_agencies)} Selected Agencies")
920
921         # Use color if 10 or fewer agencies, otherwise use grayscale for readability
922         use_color = len(selected_agencies) <= 10
923
924         if use_color:
925             fig = px.line(
926                 time_df,
927                 x="Year",
928                 y="Word Count",
929                 color="Agency",
930                 markers=True,
931                 title="Regulation Word Count Over Time",
932                 hover_data=["Agency", "Word Count"]
933             )
934         else:
935             # Create a custom plot with lighter lines for better viewing readability
936             fig = px.line(
937                 time_df,
938                 x="Year",
939                 y="Word Count",
940                 color="Agency",
941                 markers=True,
942                 title="Regulation Word Count Over Time",
943                 hover_data=["Agency", "Word Count"],
944                 color_discrete_sequence=px.colors.sequential.Greys[3:]
945             )
946             # Adjust line opacity
947             fig.update_traces(opacity=0.7, line=dict(width=1))
948
949         # Ensure X-axis shows only whole years
950         fig.update_xaxes(
951             tickmode='array',
952             tickvals=years_to_analyze,
953             ticktext=[str(year) for year in years_to_analyze],
954             dtick=1
955         )
956
957         fig.update_layout(
958             xaxis_title="Year",
959             yaxis_title="Word Count",
960             legend_title="Agency",
961             hovermode="closest"
962         )
963
964         st.plotly_chart(fig, use_container_width=True)
```

```
965
966         # Display the data table for all agencies
967         st.subheader("Word Count Data by Year and Agency")
968
969         # Pivot the data to show years as columns and agencies as rows
970         pivot_df = time_df.pivot(index="Agency", columns="Year", values="Word Count")
971         pivot_df = pivot_df.reset_index()
972
973         # Format the table for display
974         st.dataframe(pivot_df, use_container_width=True)
975
976         # Add a download button for all selected agencies data
977         all_csv = time_df.to_csv(index=False)
978         st.download_button(
979             label="Download data as CSV",
980             data=all_csv,
981             file_name=f"wordcount_agencies_{start_year}-{end_year}.csv",
982             mime="text/csv"
983         )
984     else:
985         st.info("Please select at least one agency to analyze.")
986 else:
987     st.warning("No agency data available. Please run the analysis in the 'Agency' tab.")
988
989 with tab5:
990     st.header("Agency Regulation Composition by Title")
991
992     # Check if we have the necessary data
993     if 'agencies' not in st.session_state:
994         with st.spinner("Fetching agency data..."):
995             st.session_state.agencies = get_agencies(force_refresh=not cache_results)
996
997     if st.session_state.agencies:
998         # Extract all agency names for the dropdown
999         all_agency_names = []
1000
1001     def collect_agency_names(agency_list):
1002         for agency in agency_list:
1003             all_agency_names.append(agency["name"])
1004             if "children" in agency and agency["children"]:
1005                 collect_agency_names(agency["children"])
1006
1007     collect_agency_names(st.session_state.agencies)
1008     all_agency_names = sorted(all_agency_names)
1009
1010     # Agency selection dropdown
1011     selected_agency = st.selectbox(
1012         "Select an agency to analyze its regulatory composition:",
1013         options=all_agency_names
```

```

1014         )
1015
1016         # Add note about title search scope
1017         st.info("""
1018         **Note:** This analysis searches all titles regulated by the selected agency and :
1019         "Number of titles to process" limit in the sidebar configuration. Results will inc
1020         applicable titles found in the eCFR API.
1021         """)
1022
1023         if st.button("Analyze Regulatory Composition", type="primary"):
1024             # Create a cache key based on the parameters
1025             cache_key = f"agency_composition_{selected_agency.replace(' ', '_')}_{{target_c
1026
1027             # Check if we have cached results and caching is enabled
1028             if cache_results and cache_key in st.session_state:
1029                 st.success("Using cached results. Toggle 'Cache results' off to force rec
1030
1031                 # Retrieve cached data
1032                 title_word_counts = st.session_state[f"{cache_key}_title_word_counts"]
1033                 composition_df = st.session_state[f"{cache_key}_composition_df"]
1034                 total_words = st.session_state[f"{cache_key}_total_words"]
1035                 title_fetch_errors = st.session_state.get(f"{cache_key}_title_fetch_errors
1036                 has_data = st.session_state.get(f"{cache_key}_has_data", False)
1037             else:
1038                 # Check if titles data is available or fetch it
1039                 if 'titles_info' not in st.session_state:
1040                     with st.spinner("Fetching titles data..."):
1041                         st.session_state.titles_info = get_titles(force_refresh=not cache
1042
1043                 # Get the titles information
1044                 titles_info = st.session_state.titles_info
1045
1046                 # Create agency-title mappings
1047                 agency_to_titles = create_agency_title_mapping(st.session_state.agencies)
1048                 titles_for_agency = agency_to_titles.get(selected_agency, set())
1049
1050                 has_data = False
1051
1052                 if not titles_for_agency:
1053                     st.warning(f"No title references found for {selected_agency}. This ag
1054                     composition_df = pd.DataFrame()
1055                     title_word_counts = {}
1056                     title_fetch_errors = {}
1057                     total_words = 0
1058                 else:
1059                     # Use current date for analysis (or the date from the sidebar)
1060                     target_date = target_date_str
1061
1062                     # Process only the titles relevant to this agency

```

```
1063         with st.spinner(f"Analyzing regulatory composition for {selected_agency}"):
1064             # Show progress
1065             progress_bar = st.progress(0)
1066             status_text = st.empty()
1067
1068             # Dictionary to store word counts by title
1069             title_word_counts = {}
1070             title_fetch_errors = {}
1071
1072             # Only process titles relevant to this agency
1073             relevant_titles = [t for t in titles_info if str(t["number"]) in selected_agency]
1074
1075             # If no relevant titles found, try processing all titles (fallback)
1076             if not relevant_titles:
1077                 for title_num in titles_for_agency:
1078                     # Find the title info or create a placeholder
1079                     title_match = next((t for t in titles_info if str(t["number"]) == title_num), None)
1080                     if title_match:
1081                         relevant_titles.append(title_match)
1082
1083             # If we still have no titles, process all titles as a fallback
1084             if not relevant_titles:
1085                 relevant_titles = titles_info[:10] # Limit to first 10 titles
1086
1087             for i, title_info in enumerate(relevant_titles):
1088                 title_number = title_info["number"]
1089                 title_name = title_info.get("name", "Unknown")
1090                 latest_date = title_info["latest_amended_on"]
1091
1092                 # Update progress
1093                 progress = (i + 1) / len(relevant_titles)
1094                 progress_bar.progress(progress)
1095                 status_text.write(f"Processing title {i+1} of {len(relevant_titles)}")
1096
1097                 # Skip reserved titles
1098                 if title_info.get("reserved", False):
1099                     continue
1100
1101                 # Determine date to use
1102                 use_date = target_date
1103                 if datetime.strptime(latest_date, "%Y-%m-%d") < datetime.strptime(target_date, "%Y-%m-%d"):
1104                     use_date = latest_date
1105
1106                 try:
1107                     # Fetch and process content with cache control
1108                     title_content = get_title_content(title_number, use_date,
1109                                                         headers={"Cache-Control": "no-cache"})
1110                     if title_content:
1111                         word_count = count_words_in_xml(title_content)
```

```

1112         title_word_counts[title_number] = word_count
1113     else:
1114         title_fetch_errors[title_number] = "No content received"
1115     except Exception as e:
1116         title_fetch_errors[title_number] = str(e)
1117
1118     # Add delay to avoid rate limiting
1119     time.sleep(throttle_delay)
1120
1121     # Clear progress indicators
1122     progress_bar.empty()
1123     status_text.empty()
1124
1125     # Create DataFrame for visualization if we have data
1126     composition_data = []
1127     total_words = sum(title_word_counts.values()) if title_word_counts else 0
1128
1129     if title_word_counts:
1130         has_data = True
1131         for title_num, word_count in title_word_counts.items():
1132             # Find title name
1133             title_name = next((t["name"] for t in titles_info if t["num"] == title_num), None)
1134
1135             # Calculate percentage
1136             percentage = (word_count / total_words) * 100 if total_words else 0
1137
1138             composition_data.append({
1139                 "Title Number": title_num,
1140                 "Title": f"Title {title_num}: {title_name}",
1141                 "Word Count": word_count,
1142                 "Percentage": round(percentage, 2)
1143             })
1144
1145     # Create DataFrame
1146     composition_df = pd.DataFrame(composition_data)
1147     composition_df = composition_df.sort_values(by="Percentage", ascending=False)
1148     else:
1149         composition_df = pd.DataFrame()
1150
1151     # Cache results if enabled
1152     if cache_results:
1153         st.session_state[f"{cache_key}_title_word_counts"] = title_word_counts
1154         st.session_state[f"{cache_key}_composition_df"] = composition_df
1155         st.session_state[f"{cache_key}_total_words"] = total_words
1156         st.session_state[f"{cache_key}_title_fetch_errors"] = title_fetch_errors
1157         st.session_state[f"{cache_key}_has_data"] = has_data
1158     # Store the cache key itself for reference
1159     st.session_state[cache_key] = True
1160

```



```
1161 # Display results
1162 if has_data and title_word_counts and not composition_df.empty:
1163     st.subheader(f"Regulatory Composition for {selected_agency}")
1164     st.write(f"Total Word Count: {total_words:,}")
1165
1166     # Create visualization - Pie Chart
1167     st.subheader("Percentage Breakdown by Title")
1168
1169     # For better visualization, group small slices into "Other"
1170     viz_df = composition_df.copy()
1171     threshold = 2.0 # Percentage threshold for "Other" category
1172
1173     if len(viz_df) > 10: # Only group if there are many titles
1174         main_titles = viz_df[viz_df["Percentage"] >= threshold]
1175         other_titles = viz_df[viz_df["Percentage"] < threshold]
1176
1177         if not other_titles.empty:
1178             other_sum = other_titles["Word Count"].sum()
1179             other_percentage = (other_sum / total_words) * 100
1180
1181             # Create a row for "Other"
1182             other_row = pd.DataFrame([
1183                 "Title Number": 0,
1184                 "Title": "Other Titles",
1185                 "Word Count": other_sum,
1186                 "Percentage": round(other_percentage, 2)
1187             ])
1188
1189             viz_df = pd.concat([main_titles, other_row]).reset_index(drop=True)
1190
1191     # Create the pie chart
1192     fig = px.pie(
1193         viz_df,
1194         values="Percentage",
1195         names="Title",
1196         title=f"Regulatory Composition for {selected_agency}",
1197         hover_data=["Word Count"],
1198         labels={"Percentage": "% of Total"},
1199     )
1200
1201     fig.update_traces(
1202         textposition='inside',
1203         textinfo='percent+label',
1204         insidetextorientation='radial'
1205     )
1206
1207     st.plotly_chart(fig, use_container_width=True)
1208
1209     # Create a bar chart for better comparison - WITHOUT color heat bar
```

```

1210         fig2 = px.bar(
1211             viz_df,
1212             x="Title",
1213             y="Percentage",
1214             title=f"Regulatory Composition for {selected_agency} (Bar Chart)",
1215             hover_data=["Word Count"],
1216             color_discrete_sequence=["#1f77b4"] # Use a single color for all bars
1217         )
1218
1219         fig2.update_layout(
1220             xaxis_title="Title",
1221             yaxis_title="Percentage of Total (%)",
1222             xaxis={'categoryorder': 'total descending'}
1223         )
1224
1225         # Add percentage labels on top of the bars
1226         fig2.update_traces(
1227             texttemplate='%{y:.1f}%',
1228             textposition='outside'
1229         )
1230
1231         st.plotly_chart(fig2, use_container_width=True)
1232
1233         # Display the full data table
1234         st.subheader("Complete Breakdown by Title")
1235         st.dataframe(composition_df[["Title", "Word Count", "Percentage"]], use_co
1236
1237         # Add download button for results
1238         csv = composition_df.to_csv(index=False)
1239         st.download_button(
1240             label=f"Download composition data for {selected_agency}",
1241             data=csv,
1242             file_name=f"regulatory_composition_{selected_agency.replace(' ', '_')}.csv",
1243             mime="text/csv"
1244         )
1245     else:
1246         error_message = "No word count data could be obtained for the titles regu
1247         st.warning(error_message)
1248
1249         # Show more detailed error information
1250         if title_fetch_errors:
1251             st.error("Details on errors encountered:")
1252
1253             error_df = pd.DataFrame([
1254                 {"Title": f"Title {title}", "Error": error}
1255                 for title, error in title_fetch_errors.items()
1256             ])
1257
1258             if not error_df.empty:

```

```
1259         st.dataframe(error_df, use_container_width=True)
1260
1261         # Suggest solutions
1262         st.info("""
1263         **Possible solutions:**
1264         1. Try a different agency that regulates more common titles
1265         2. Check the API connection by using other tabs first
1266         3. Increase the API request delay in the advanced options to avoid rate l
1267         4. Try with a different target date
1268         """)
1269     else:
1270         st.warning("No agency data available. Please run the analysis in the 'Agency Analy
1271
1272 with tab6:
1273     st.header("About This App")
1274
1275     st.markdown("""
1276     ## Federal Regulations Word Count Analysis Tool
1277
1278     This application analyzes the Electronic Code of Federal Regulations (eCFR) to calculate
1279     """)
1280
1281     st.subheader("Key Features")
1282
1283     col1, col2 = st.columns(2)
1284
1285     with col1:
1286         st.markdown("#### 1. Agency Analysis")
1287         st.markdown("""
1288         - Calculates word counts for all federal agencies
1289         - Visualizes the top 10 agencies by word count
1290         - Provides downloadable CSV data
1291         """)
1292
1293         st.markdown("#### 3. Agency Hierarchy Visualization")
1294         st.markdown("""
1295         - Interactive sunburst chart showing agency relationships
1296         - Filtering options for parent agencies and independent agencies
1297         - Tabular view of agency hierarchical structure
1298         """)
1299
1300         st.markdown("#### 5. Regulatory Composition")
1301         st.markdown("""
1302         - Analyzes the distribution of an agency's regulations across different titles
1303         - Provides pie charts and bar charts showing percentage breakdowns
1304         - Helps identify which titles contribute most to an agency's regulatory footprint
1305         """)
1306
1307     with col2:
```

```

1308     st.markdown("#### 2. Title Analysis")
1309     st.markdown("""
1310     - Displays word counts for each CFR title
1311     - Identifies the largest titles by word count
1312     - Offers visualization and downloadable data
1313     """)
1314
1315     st.markdown("#### 4. Word Count Over Time")
1316     st.markdown("""
1317     - Tracks changes in regulatory word counts across multiple years
1318     - Allows selection of specific agencies for time series comparison
1319     - Interactive line chart with data download options
1320     """)
1321
1322     st.subheader("Technical Capabilities")
1323
1324     tech_col1, tech_col2 = st.columns(2)
1325
1326     with tech_col1:
1327         st.markdown("""
1328         - Caching System: Stores results to improve performance and reduce API calls
1329         - Throttling Controls: Adjustable delay between API requests to prevent rate limit
1330         """)
1331
1332     with tech_col2:
1333         st.markdown("""
1334         - Error Handling: Robust handling of timeouts and problematic titles
1335         - Advanced Configuration: Customizable timeout settings and title processing options
1336         """)
1337
1338     st.subheader("Understanding the CFR")
1339
1340     st.markdown("""
1341     The Code of Federal Regulations (CFR) is divided into 50 titles that represent broad areas of
1342
1343     - Titles 1-16: General Administrative Functions
1344     - Titles 17-27: Economic Regulation
1345     - Titles 28-41: Labor and National Defense
1346     - Titles 42-50: Public Health and Welfare
1347
1348     Each title contains regulations from multiple agencies. When a title is regulated by multiple
1349     """)
1350
1351     st.subheader("Data Processing")
1352
1353     st.markdown("""
1354     1. The app connects to the eCFR API to fetch data about agencies, titles, and regulations
1355     2. It creates mappings between agencies and the titles they regulate
1356     3. For each title, it retrieves the full XML content as of the selected date
1357     """)

```

```
1357 4. It parses the XML content using BeautifulSoup to calculate accurate word counts
1358 5. It allocates word counts to agencies based on the titles they regulate
1359 6. Results are visualized using Matplotlib and Plotly
1360 """)
1361
1362 st.subheader("Usage Tips")
1363
1364 st.markdown("""
1365 - Use the sidebar to configure analysis parameters
1366 - Increase the API request delay for more reliable results
1367 - Enable "Skip known large titles" option if experiencing timeouts
1368 - The cache feature speeds up repeated analyses with the same parameters
1369 """)
1370
1371 st.subheader("Data Attribution")
1372
1373 st.markdown("""
1374 All regulation data is sourced from the [eCFR API](https://www.ecfr.gov) provided by the
1375 """)
1376
1377 if __name__ == "__main__":
1378     main()
```